

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

OPERATING SYSTEMS (23CS4PCOPS)

Submitted by

**Arjun Mallikarjun Banappanavar
(1BM24CS052)**

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**OPERATING SYSTEMS**” carried out by **Arjun Mallikarjun Banappanavar(1BM24CS052)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Operating Systems Lab (**23CS4PCOPS**) work prescribed for the said degree.

Praveen N
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. a) FCFS b) SJF c) Priority d) Round Robin	1,5,9, 15
2	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	19
3	Write a C program to simulate Real-Time CPU Scheduling algorithms: a) Rate-Monotonic b) Earliest-deadline First c) Proportional scheduling	28,33 ,38
4	Write a C program to simulate: a) Producer-Consumer problem using semaphores. b) Dining-Philosopher's problem	41,45
5	Write a C program to simulate: a) Banker's algorithm for the purpose of deadlock avoidance. b) Deadlock Detection	49,54
6	Write a C program to simulate the following contiguous memory allocation techniques. a) Worst-fit b) Best-fit c) First-fit	57
7	Write a C program to simulate page replacement algorithms. a) FIFO b) LRU c) Optimal	63

Course outcomes:

CO1	Apply the different concepts and functionalities of Operating System.
CO2	Analyse various Operating system strategies and techniques.
CO3	Demonstrate the different functionalities of Operating System
CO4	Conduct practical experiments to implement the functionalities of Operating system.

LAB PROGRAM 1A:

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.

FCFS:

Code:

```
#include <stdio.h>

void swap(int *xp, int *yp)
{
    int temp = *xp;
    *xp = *yp;
    *yp = temp;
}

void bubbleSort(int arr[], int n)
{
    int i, j;
    int swapped;
    for(i = 0; i < n - 1; i++)
    {
        swapped = 0;
        for(j = 0; j < n - i - 1; j++)
        {
            if(arr[j] > arr[j + 1])
            {
                swap(&arr[j], &arr[j + 1]);
                swapped = 1;
            }
        }
        if(swapped == 0)
            break;
    }
}
```

```

int main()
{
    int n, ct[20], tat[20], current_time = 0;
    int wt[20], sumwt = 0, sumtat = 0;
    printf("Enter the number of processes:");
    scanf("%d", &n);
    int at[20], bt[20], pid[20];
    printf("Enter the AT and BT\n");
    for(int i = 0; i < n; i++)
    {
        scanf("%d", &at[i]);
        scanf("%d", &bt[i]);
        pid[i] = i + 1;
    }
    bubbleSort(at, n);
    for(int i = 0; i < n; i++)
    {
        if(current_time < at[i])
            current_time = at[i];
        ct[i] = current_time + bt[i];
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
        current_time = ct[i];
        sumwt += wt[i];
        sumtat += tat[i];
    }
    int awt = sumwt / n;
    int atat = sumtat / n;
    printf("PID\tAT\tBT\tCT\tWT\tTAT\n");

```

```

for(int i = 0; i < n; i++)
{
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",
        pid[i], at[i], bt[i],
        ct[i], wt[i], tat[i]);
}
printf("AWT = %d\n", awt);
printf("ATAT = %d\n", atat);
return 0;
}

```

Output:

```

F:\os_lab\fcfs.exe
Enter the number of processes:2
Enter the AT and BT
10 20 10 20
PID    AT    BT    CT    WT    TAT
1      10    20    30    0     20
2      10    20    50    20    40
AWT = 10
ATAT = 30

Process returned 0 (0x0)   execution time : 26.581 s
Press any key to continue.

```

Date: / /
Page:

OS LAB

① FCFS

```
#include <stdio.h>

void swap(int *xp, int *yp) {
    int temp = *xp;
    *xp = *yp;
    *yp = temp;
}

void bubbleSort(int arr[], int n) {
    int i, j;
    int swapped;
    for (i = 0; i < n-1; i++) {
        swapped = 0;
        for (j = 0; j < n-i-1; j++) {
            if (arr[j] > arr[j+1]) {
                swap(&arr[j], &arr[j+1]);
                swapped = 1;
            }
        }
        if (!swapped) break;
    }
}

void main() {
    int n, ct[10], tat[10], current_time = 0;
    int wt[10], sumwt, sumtat = 0;
    printf("Enter the no. of processes: ");
    scanf("%d", &n);
    int at[n], bt[n], pid[n];
```

Date: / /
Page:

```
printf("Enter the at & bt\n");
for (int i = 0; i < n; i++) {
    scanf("%d", &at[i]);
    scanf("%d", &bt[i]);
    pid[i] = i+1;
}

bubbleSort(at, n);
for (int i = 0; i < n; i++) {
    if (current_time < at[i]) {
        current_time = at[i];
    }
    ct[i] = current_time - at[i];
    tat[i] = ct[i] + bt[i];
    wt[i] = tat[i] - bt[i];
    sumwt = wt[i];
    sumtat = tat[i];
}

int awt = sumwt/n;
int atat = sumtat/n;
printf("These are the corresponding times:\n");
for (int i = 0; i < n; i++) {
    printf("pid: %d, at: %d, bt: %d, ct: %d, tat: %d, wt: %d\n",
           pid[i], at[i], bt[i], ct[i], tat[i], wt[i]);
}
```

Output:

Enter the number of processes: 2
 Enter the at & bt: 10 20 10 20
 These are the pid and corresponding time

pid	at	bt	ct	tat	wt
1	10	20	0	20	20
2	10	20	0	20	20

Output:

Enter the number of processes: 2
 Enter the at & bt: 0 10 10 20
 These are the pid and corresponding times

pid	at	bt	ct	tat	wt
1	0	10	0	10	10
2	10	20	0	20	20

LAB PROGRAM 1B:

SJF:

Code:

```
#include <stdio.h>

int main()
{
    int n, i, j;
    int at[20], bt[20], tat[20], wt[20];
    int comp[20] = {0};

    int current_time = 0, completed_count = 0;
    int minbt, index;
    float avg_wt = 0;
    float avg_tat = 0;
    printf("Enter number of processes:");
    scanf("%d", &n);
    for(i = 0; i < n; i++)
    {
        printf("Enter AT and BT for P%d:", i + 1);
        scanf("%d%d", &at[i], &bt[i]);
    }
    while(completed_count < n)
    {
        minbt = 9999;
        index = -1;
        for(i = 0; i < n; i++)
        {
            if(at[i] <= current_time && comp[i] == 0)
            {
                if(bt[i] < minbt)
```



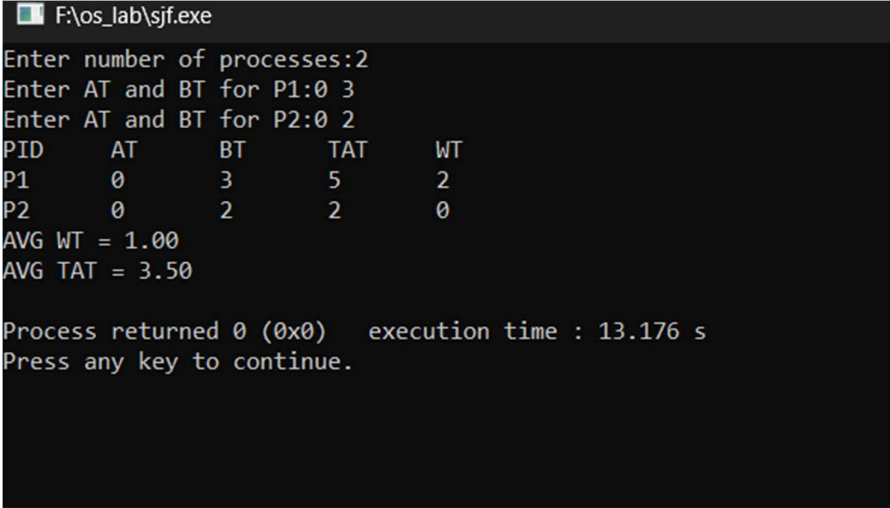
```

        {
            minbt = bt[i];
            index = i;
        }
    }
}
if(index == -1)
{
    current_time++;
}
else
{
    int ct = current_time + bt[index];
    tat[index] = ct - at[index];
    wt[index] = tat[index] - bt[index];
    current_time = ct;
    comp[index] = 1;
    completed_count++;
    avg_wt += wt[index];
    avg_tat += tat[index];
}
}
avg_wt /= n;
avg_tat /= n;
printf("PID\tAT\tBT\tTAT\tWT\n");
for(i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], tat[i], wt[i]);
}
printf("AVG WT = %.2f\n", avg_wt);

```

```
printf("AVG TAT = %.2f\n", avg_tat);  
return 0;  
}
```

Output:



```
F:\os_lab\sjf.exe  
Enter number of processes:2  
Enter AT and BT for P1:0 3  
Enter AT and BT for P2:0 2  
PID    AT    BT    TAT    WT  
P1      0     3     5     2  
P2      0     2     2     0  
AVG WT = 1.00  
AVG TAT = 3.50  
  
Process returned 0 (0x0)   execution time : 13.176 s  
Press any key to continue.
```

Q. 33P

```

#include <stdio.h>

int main() {
    int n, i, j;
    int at[20], bt[20], tat[20], ct[20];
    int comp[20] = {0};
    int current_time = 0, completed_count = 0;
    int min_bt, index;
    float avg_wt = 0;
    float avg_tat = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for(i = 0; i < n; i++) {
        printf("Enter AT and BT for p%d: ", i+1);
        scanf("%d %d", &at[i], &bt[i]);
    }
    while(comp[0] < n) {
        min_bt = 9999;
        index = -1;
        for(i = 0; i < n; i++) {
            if(at[i] <= current_time ||
               comp[i] == 0) {
                if(bt[i] < min_bt) {
                    min_bt = bt[i];
                    index = i;
                }
            }
        }
        if(index == -1)
            current_time++;
    }
}

```

```

else {
    ct[index] = current_time + bt[index];
    tat[index] = ct[index] - at[index];
    wt[index] = tat[index] - bt[index];
    current_time = ct[index];
    comp[index] = 1;
    completed_count++;
    avg_wt += wt[index];
    avg_tat += tat[index];
}
avg_wt /= n;
avg_tat /= n;
printf("AT BT CT TAT WT\n");
for(i = 0; i < n; i++) {
    printf("p%d\t", i+1);
    printf("%d\t", at[i]);
    printf("%d\t", bt[i]);
    printf("%d\t", ct[i]);
    printf("%d\t", tat[i]);
    printf("%d\t", wt[i]);
    printf("\n");
}
printf("AV WT = %f, AV TAT = %f", avg_wt, avg_tat);
return 0;
}

```

Output:

Case 1:

Enter number of processes: 2

Enter AT and BT for P1: 0 5

Enter AT and BT for P2: 1 4

PID	AT	BT	CT	TAT	WT
P1	0	5	5	5	0
P2	1	4	9	8	4

AV WT = 2.00
AV TAT = 6.5

Case 2:

Enter number of processes: 2

Enter AT and BT for P1: 0 3

Enter AT and BT for P2: 0 2

PID	AT	BT	CT	TAT	WT
P1	0	3	3	3	0
P2	0	2	2	2	0

AV WT = 1.00
AV TAT = 2.50

17/10/2020

LAB PROGRM 1C:

PRIORITY SCHEDULING:

Code:

```
#include <stdio.h>

void nonpre( int n, int *pid, int *at, int *bt, int *priority, int *ct,int *tat, int *wt,int
*completed)
{
    int time = 0, count = 0;
    while(count < n)
    {
        int idx = -1;
        for(int i = 0; i < n; i++)
        {
            if(at[i] <= time && completed[i] == 0)
            {
                if(idx == -1 || priority[i] < priority[idx])
                    idx = i;
            }
        }
        if(idx != -1)
        {
            time += bt[idx];
            ct[idx] = time;
            tat[idx] = ct[idx] - at[idx];
            wt[idx] = tat[idx] - bt[idx];
            completed[idx] = 1;
            count++;
        }
        else
        {
            time++;
        }
    }
}
```

```

    }
}
}
void pre( int n,int *pid,int *at, int *bt,int *priority, int *ct,int *tat,int *wt,int *completed)
{
    int time = 0, count = 0;
    int rt[n];
    for(int i = 0; i < n; i++)
    {
        rt[i] = bt[i];
        completed[i] = 0;
    }
    while(count < n)
    {
        int idx = -1;
        for(int i = 0; i < n; i++)
        {
            if(at[i] <= time && completed[i] == 0)
            {
                if(idx == -1 || priority[i] < priority[idx])
                    idx = i;
            }
        }
        if(idx == -1)
        {
            time++;
        }
        else
        {
            rt[idx]--;

```

```

        time++;
        if(rt[idx] == 0)
        {
            ct[idx] = time;
            tat[idx] = ct[idx] - at[idx];
            wt[idx] = tat[idx] - bt[idx];
            completed[idx] = 1;
            count++;
        }
    }
}

int main()
{
    int n;
    printf("Enter number of processes:");
    scanf("%d", &n);
    int at[n], bt[n], pri[n];
    int pid[n], ct[n], tat[n], wt[n], comp[n];
    int ch;
    printf("1. Non Preemptive\n2. Preemptive\n");
    scanf("%d", &ch);
    for(int i = 0; i < n; i++)
    {
        printf("AT BT Priority for Process %d:\n", i + 1);
        scanf("%d%d%d",
            &at[i],
            &bt[i],
            &pri[i]);
        comp[i] = 0;
    }
}

```

```

        pid[i] = i + 1;
    }
    switch(ch)
    {
        case 1:
            nonpre(n,pid,at,bt,pri,ct,tat,wt,comp);
            break;
        case 2:
            pre(n,pid,at,bt,pri,ct,tat,wt,comp);
            break;
        default:
            printf("Invalid");
            return 1;
    }
    float atat = 0, awt = 0;
    for(int i = 0; i < n; i++)
    {
        atat += tat[i];
        awt += wt[i];
    }
    atat /= n;
    awt /= n;
    printf("PID\tAT\tBT\tPRI\tCT\tTAT\tWT\n");
    for(int i = 0; i < n; i++)
    {
        printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
            pid[i], at[i], bt[i],
            pri[i], ct[i],
            tat[i], wt[i]);
    }

```

```

printf("ATAT=%f\n", atat);

printf("AWT=%f\n", awt);

return 0;

}

```

Output:

```

F:\os_lab\priority.exe
Enter number of processes:3
1. Non Preemptive
2. Preemptive
2
AT BT Priority for Process 1:
0 3 3
AT BT Priority for Process 2:
1 5 1
AT BT Priority for Process 3:
3 3 2

```

PID	AT	BT	PRI	CT	TAT	WT
1	0	3	3	11	11	8
2	1	5	1	6	5	0
3	3	3	2	9	6	3

```

ATAT=7.333333
AWT=3.666667

Process returned 0 (0x0)  execution time : 16.959 s
Press any key to continue.

```

```

F:\os_lab\priority.exe
Enter number of processes:3
1. Non Preemptive
2. Preemptive
1
AT BT Priority for Process 1:
0 3 3
AT BT Priority for Process 2:
1 5 1
AT BT Priority for Process 3:
3 3 2

```

PID	AT	BT	PRI	CT	TAT	WT
1	0	3	3	3	3	0
2	1	5	1	8	7	2
3	3	3	2	11	8	5

```

ATAT=6.000000
AWT=2.333333

Process returned 0 (0x0)  execution time : 27.120 s
Press any key to continue.

```


LAB PROGRAM 1D:

ROUND ROBIN SCHEDULING:

Code:

```
#include <stdio.h>

void roundrobin(int n,int *at,int *bt,int *rt,int *ct,int tq)
{
    int time = 0, comp = 0;
    int vis[n];
    for(int i = 0; i < n; i++)
        vis[i] = 0;
    while(comp < n)
    {
        int found = 0;
        for(int i = 0; i < n; i++)
        {
            if(at[i] <= time && rt[i] > 0)
            {
                found = 1;
                if(rt[i] > tq)
                {
                    time += tq;
                    rt[i] -= tq;
                }
            }
            else
            {
                time += rt[i];
                rt[i] = 0;
                ct[i] = time;
                comp++;
            }
        }
    }
}
```

```

        }
    }
    if(found == 0)
        time++;
    }
}

int main()
{
    int n, tq;
    printf("Enter number of processes:");
    scanf("%d", &n);
    int pid[n], at[n], bt[n], rt[n], ct[n];
    printf("Enter quantum time:");
    scanf("%d", &tq);
    for(int i = 0; i < n; i++)
    {
        pid[i] = i + 1;
        printf("Process %d\n", pid[i]);
        scanf("%d", &at[i]);
        scanf("%d", &bt[i]);
        rt[i] = bt[i];
    }
    roundrobin(n, at, bt, rt, ct, tq);
    printf("PID\tAT\tBT\tCT\n");
    for(int i = 0; i < n; i++)
    {
        printf("%d\t%d\t%d\t%d\n",
            pid[i], at[i], bt[i], ct[i]);
    }
    return 0;
}

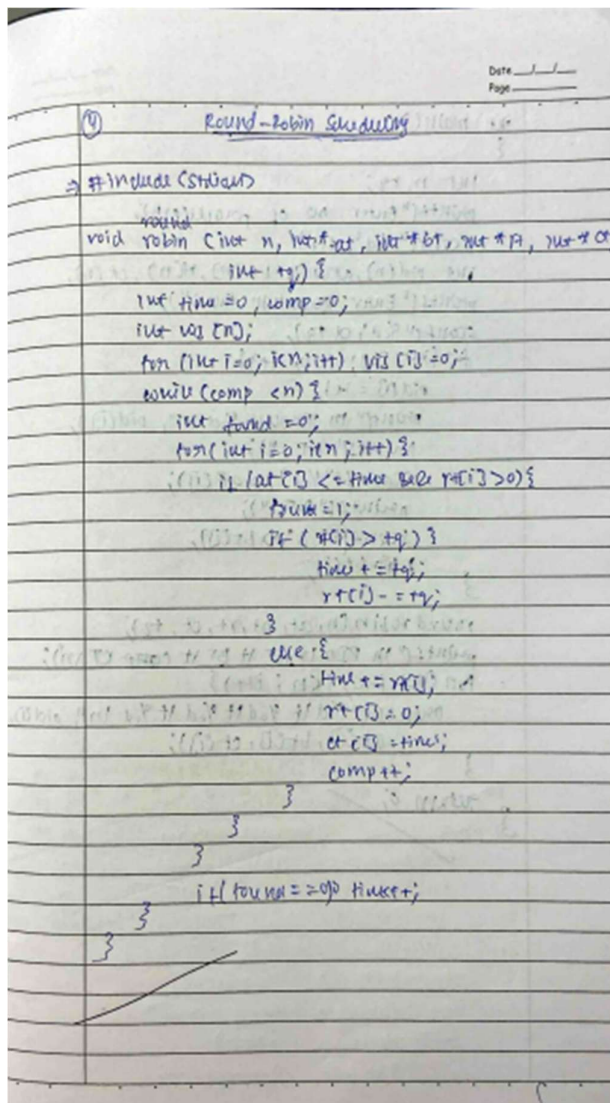
```

}

Output:

```
F:\os_lab\roundrobin.exe
Enter number of processes:3
Enter quantum time:2
Process 1
0 20
Process 2
2 5
Process 3
8 6
PID    AT    BT    CT
1       0    20    31
2       2     5    13
3       8     6    19

Process returned 0 (0x0)    execution time : 16.242 s
Press any key to continue.
```



```

int main()
{
    int n, m;
    printf("Enter no of processes\n");
    scanf("%d", &n);
    int pid[n], at[n], bt[n], rt[n], ct[n];
    printf("Enter quantum time\n");
    scanf("%d", &rt);
    for (int i = 0; i < n; i++)
    {
        pid[i] = i+1;
        printf("Process %d\n", pid[i]);
        printf("AT: ");
        scanf("%d", &at[i]);
        printf("BT: ");
        scanf("%d", &bt[i]);
        rt[i] = rt;
    }
    roundRobin(n, at, bt, rt, ct, rt);
    printf("in PID: AT: BT: RT: CT: WT:\n");
    for (int i = 0; i < n; i++)
    {
        printf("%d\t%d\t%d\t%d\t%d\t%d\n",
            pid[i], at[i], bt[i], rt[i], ct[i], wt[i]);
    }
    return 0;
}

```

OUTPUT :

Enter no of processes: 3

Enter quantum time: 2

Process 1

AT: 0

BT: 20

Process 2

AT: 2

BT: 5

Process 3

AT: 8

BT: 6

PID	AT	BT	CT	WT	RT
1	0	20	31	31	11
2	2	5	13	11	6
3	8	6	19	11	5

total = 12.66666

avg = 7.33333

PID	AT	BT	CT	WT	RT
P1	0	20	31	31	11
P2	2	5	13	11	6
P3	8	6	19	11	5

total = 6.

LAB PROGRAM 2:

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

Code:

```
#include <stdio.h>

#include <string.h>

#define MAX 100

typedef struct
{
    int pid;

    int arrival;

    int burst;

    int remaining;

    int completion;

    int waiting;

    int turnaround;

    char type[10];
} Process;

typedef struct
{
    int items[MAX];

    int front, rear;
} Queue;

void initQueue(Queue *q)
{

```

```

    q->front = q->rear = -1;
}

int isEmpty(Queue *q)
{
    return (q->front == -1);
}

void enqueue(Queue *q, int v)
{
    if(q->rear == MAX - 1)
        return;

    if(q->front == -1)
        q->front = 0;

    q->items[++q->rear] = v;
}

int dequeue(Queue *q)
{
    if(isEmpty(q))
        return -1;

    int val = q->items[q->front];

    if(q->front == q->rear)
        q->front = q->rear = -1;

    else
        q->front++;

    return val;
}

```

```

void sortByArrival(Process p[], int n)
{
    Process temp;
    for(int i = 0; i < n - 1; i++)
    {
        for(int j = 0; j < n - i - 1; j++)
        {
            if(p[j].arrival > p[j + 1].arrival)
            {
                temp = p[j];
                p[j] = p[j + 1];
                p[j + 1] = temp;
            }
        }
    }
}

void mlq()
{
    int n;

    Process p[MAX];

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(int i = 0; i < n; i++)
    {
        printf("\nProcess %d\n", i + 1);
    }
}

```



```

    printf("Arrival Time: ");

    scanf("%d", &p[i].arrival);

    printf("Burst Time: ");

    scanf("%d", &p[i].burst);

    printf("Type(system/user): ");

    scanf("%s", p[i].type);

    p[i].pid = i + 1;

    p[i].remaining = p[i].burst;
}

sortByArrival(p, n);

Queue systemQ, userQ;

initQueue(&systemQ);

initQueue(&userQ);

int time = 0;

int completed = 0;

int i = 0;

int current = -1;

int gantt[MAX];

int gtime[MAX];

int gindex = 0;

while(completed < n)
{
    while(i < n && p[i].arrival <= time)
    {
        if(strcmp(p[i].type, "system") == 0)

```

```

        enqueue(&systemQ, i);
    else
        enqueue(&userQ, i);
    i++;
}
if(current == -1)
{
    if(!isEmpty(&systemQ))
        current = dequeue(&systemQ);
    else if(!isEmpty(&userQ))
        current = dequeue(&userQ);
    else
    {
        time++;
        continue;
    }
}
gantt[gindex] = p[current].pid;
gtime[gindex] = time;
gindex++;
p[current].remaining--;
time++;
if(p[current].remaining == 0)
{
    p[current].completion = time;

```

```

        completed++;

        current = -1;

    }
}

gtime[gindex] = time;

float twt = 0;

float ttat = 0;

for(int i = 0; i < n; i++)
{
    p[i].turnaround =

        p[i].completion - p[i].arrival;

    p[i].waiting =

        p[i].turnaround - p[i].burst;

    twt += p[i].waiting;

    ttat += p[i].turnaround;

}

printf("\nGantt Chart:\n");

for(int i = 0; i < gindex; i++)

    printf("P%d ", gantt[i]);

printf("\n");

for(int i = 0; i <= gindex; i++)

    printf("%d ", gtime[i]);

printf("\n");

printf("\nPID\tTYPE\tAT\tBT\tCT\tTAT\tWT\n");

for(int i = 0; i < n; i++)

```

```

    {
        printf("%d\t%s\t%d\t%d\t%d\t%d\t%d\n",
            p[i].pid,
            p[i].type,
            p[i].arrival,
            p[i].burst,
            p[i].completion,
            p[i].turnaround,
            p[i].waiting);
    }

    printf("AWT = %.2f\n", twt / n);
    printf("ATAT = %.2f\n", ttat / n);
}

int main()
{
    mlq();

    return 0;
}

```

Output:

```
F:\os_lab\multilevelqueue.exe
Enter number of processes: 3

Process 1
Arrival Time: 0
Burst Time: 5
Type(system/user): system

Process 2
Arrival Time: 2
Burst Time: 6
Type(system/user): system

Process 3
Arrival Time: 3
Burst Time: 8
Type(system/user): user

Gantt Chart:
P1 P1 P1 P1 P1 P2 P2 P2 P2 P2 P2 P3 P3 P3 P3 P3 P3 P3 P3
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19

PID    TYPE    AT    BT    CT    TAT    WT
1      system    0     5     5     5     0
2      system    2     6    11     9     3
3      user      3     8    19    16     8
AWT = 3.67
ATAT = 10.00

Process returned 0 (0x0)  execution time : 27.700 s
Press any key to continue.
```

[illegible]

27

LAB PROGRAM 3A:

Write a C program to simulate Real-Time CPU Scheduling algorithms:

RMS:

Code:

```
#include <stdio.h>

typedef struct
{
    int id;
    int period;
    int execution;
    int remaining;
    int deadline;
} Task;

int main()
{
    int n;

    printf("Enter number of tasks: ");
    scanf("%d", &n);

    Task t[n];

    for(int i = 0; i < n; i++)
    {
        printf("Task %d\n", i + 1);
        printf("Period: ");
        scanf("%d", &t[i].period);
        printf("Execution Time: ");
```

```

scanf("%d", &t[i].execution);

t[i].id = i + 1;

t[i].remaining = 0;

t[i].deadline = t[i].period;
}

int hyper = 1;

for(int i = 0; i < n; i++)

    hyper *= t[i].period;

printf("\nSchedule:\n");

for(int time = 0; time < hyper; time++)
{
    for(int i = 0; i < n; i++)
    {
        if(time % t[i].period == 0)
        {
            t[i].remaining = t[i].execution;

            t[i].deadline = time + t[i].period;
        }
    }

    int selected = -1;

    for(int i = 0; i < n; i++)
    {
        if(t[i].remaining > 0)
        {
            if(selected == -1 ||

```



```

        t[i].period < t[selected].period)
    {
        selected = i;
    }
}
}
if(selected != -1)
{
    printf("%d -> T%d\n",
        time,
        t[selected].id);
    t[selected].remaining--;
}
else
{
    printf("%d -> Idle\n", time);
}
}
return 0;
}

```

Output:

```

F:\os_lab\rms.exe
Enter number of tasks: 3
Task 1
Period: 4
Execution Time: 1
Task 2
Period: 5
Execution Time: 2
Task 3
Period: 10
Execution Time: 1

Schedule:
0 -> T1
1 -> T2
2 -> T2
3 -> T3
4 -> T1
5 -> T2
6 -> T2
7 -> Idle
8 -> T1
9 -> Idle
10 -> T2
11 -> T2
12 -> T1
13 -> T3
14 -> Idle
15 -> T2
16 -> T1
17 -> T2

```

⑥ RMS

```

#include <stdio.h>
#include <stdlib.h>
#include <math.h>

struct process {
    int pid, burst, period, remaining;
};

int gcd(int a, int b) {
    if (b == 0) return a;
    return gcd(b, a % b);
}

int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}

int main() {
    int n;
    printf("Enter the no of processes:");
    scanf("%d", &n);
    struct process p[n];
    printf("Enter the CPU burst times: \n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &p[i].burst);
        p[i].pid = i + 1;
        p[i].remaining = 0;
    }
    printf("LCM = %d \n", hyperPeriod);
    printf("RMS: \n");
    printf("PID\tburst\t\tperiod\t\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t%d\t\t%d\t\t", p[i].pid, p[i].burst, p[i].period);
        printf("\n");
    }
}

```

```

double h = 0.0;
for (int i = 0; i < n; i++) {
    h = (double) p[i].burst / p[i].period;
}
double bound = n * (pow(2.0, 1.0 / h) - 1);
printf("m %d <= %d => %d \n", a, bound, a <= bound ? a : bound);
printf("Scheduling occurs for %d ms \n", hyperPeriod);
int prev_chosen = -1;
for (int t = 0; t < hyperPeriod; t++) {
    for (int i = 0; i < n; i++) {
        if (t % p[i].period == 0) {
            p[i].remaining = p[i].burst;
        }
    }
    int chosen = -1;
    int min_period = 999999;
    for (int i = 0; i < n; i++) {
        if (p[i].remaining > 0 && p[i].period < min_period) {
            min_period = p[i].period;
            chosen = i;
        }
    }
    if (chosen != prev_chosen) {
        printf("Scheduling occurs for %d ms \n", min_period);
        printf("PID\tburst\t\tperiod\t\n");
        for (int i = 0; i < n; i++) {
            printf("%d\t%d\t\t%d\t\t", p[i].pid, p[i].burst, p[i].period);
            printf("\n");
        }
    }
    p[chosen].remaining -= min_period;
    prev_chosen = chosen;
}

```

```

prev_chosen = chosen;
}
if (chosen != -1) {
    p[chosen] = remaining--;
}
}
return 0;
}

```

Output:

Enter the no. of processes: 3

Enter the CPU burst time:

1 2 1

Enter the time periods:

4 5 10

LCM = 20

RMS:

PID	Burst	Period
1	1	4
2	2	5
3	1	10

$0.75 \leq 0.779763 \Rightarrow \text{true}$

scheduling for 20 ms

0 ms : P1

1 ms : P2

3 ms : idle P3

4 ms : P1

5 ms : P2

7 ms : idle

8 ms : P1

9 ms : idle

10 ms : P2

12 ms : P1

13 ms : P3

14 ms : idle

15 ms : idle P2

16 ms : P1

18 ms : idle

ganer client:

#

Time (ms)	Process
0	P1
1	P2
2	idle
3	P3
4	P1
5	P2
6	P2
7	idle
8	P1
9	P2
10	idle
11	idle
12	P1
13	P2
14	P3
15	idle
16	P1
17	P2
18	idle

LAB PROGRAM 3B:

EDF:

Code:

```
#include <stdio.h>

typedef struct
{
    int id;

    int period;

    int execution;

    int remaining;

    int deadline;
} Task;

int main()
{
    int n;

    printf("Enter number of tasks: ");

    scanf("%d", &n);

    Task t[n];

    for(int i = 0; i < n; i++)
    {
        printf("Task %d\n", i + 1);

        printf("Period: ");

        scanf("%d", &t[i].period);

        printf("Execution Time: ");

        scanf("%d", &t[i].execution);
```

```

    t[i].id = i + 1;

    t[i].remaining = 0;

    t[i].deadline = t[i].period;
}

int hyper = 1;

for(int i = 0; i < n; i++)

    hyper *= t[i].period;

printf("\nSchedule:\n");

for(int time = 0; time < hyper; time++)
{
    for(int i = 0; i < n; i++)
    {
        if(time % t[i].period == 0)
        {
            t[i].remaining = t[i].execution;

            t[i].deadline = time + t[i].period;
        }
    }

    int selected = -1;

    for(int i = 0; i < n; i++)
    {
        if(t[i].remaining > 0)
        {
            if(selected == -1 ||

                t[i].period < t[selected].period)

```

```

        {
            selected = i;
        }
    }
}
if(selected != -1)
{
    printf("%d -> T%d\n",
        time,
        t[selected].id);

    t[selected].remaining--;
}
else
{
    printf("%d -> Idle\n", time);
}
}
return 0;
}

```

Output:

```
F:\os_lab\edf.exe
Enter number of tasks: 3
Task 1
Period: 6
Execution Time: 1
Task 2
Period: 8
Execution Time: 2
Task 3
Period: 5
Execution Time: 3

Schedule:
0 -> T3
1 -> T3
2 -> T3
3 -> T1
4 -> T2
5 -> T3
6 -> T3
7 -> T3
8 -> T1
9 -> T2
10 -> T3
11 -> T3
12 -> T3
13 -> T1
14 -> T2
15 -> T3
16 -> T3
17 -> T3
```

⑤ EDF

```
#include <stdio.h>
struct Task {
    int pid, burst, deadline, period, sum, abs_d;
};

int gcd(int a, int b) {
    if (b == 0) return a;
    return gcd(b, a % b);
}

int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}

int main() {
    int n;
    printf("Enter no. of processes: ");
    scanf("%d", &n);
    struct Task task[n];
    int hyperperiod = 1;
    for (int i = 0; i < n; i++) {
        printf("Enter ID, Burst, Deadline, and Period for process %d: ", i + 1);
        scanf("%d %d %d %d", &task[i].pid, &task[i].burst, &task[i].deadline, &task[i].period);
        task[i].sum = 0;
        task[i].abs_d = 0;
        hyperperiod = lcm(hyperperiod, task[i].period);
    }
    printf("Hyperperiod is %d\n", hyperperiod);
}
```

```
for (int i = 0; i < n; i++) {
    printf("%d %d %d %d %d %d\n", task[i].pid, task[i].burst, task[i].deadline, task[i].period, task[i].sum, task[i].abs_d);
}

printf("Scheduling occurs for %d ms\n", hyperperiod);
int prev_selected = -1;
for (int i = 0; i < n; i++) {
    if (task[i].period == 0) {
        if (task[i].sum > 0) {
            printf("Process %d missed by %d ms\n", task[i].pid, task[i].sum);
            task[i].sum = task[i].burst;
            task[i].abs_d = task[i].deadline + task[i].sum;
        }
    }
    int selected = -1;
    int min_abs_d = 999999;
    for (int i = 0; i < n; i++) {
        if (task[i].sum > 0 && task[i].abs_d < min_abs_d) {
            min_abs_d = task[i].abs_d;
            selected = i;
        }
    }
    if (selected != prev_selected) {
        printf("Process %d is selected\n", task[selected].pid);
    }
}
```

```
int i;
printf("Process name and ID: CPU ID\n");
scanf("%d", &i);
```

```
prev_selected = selected;
if (selected == -1) {
    printf("No process selected\n");
}
return 0;
```

Output:

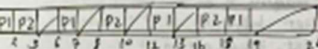
```
Enter no. of processes: 3
Enter PID, Burst, Deadline, and Period for process 1:
1 1 3 6
Process 2:
2 2 5 8
PID Burst Deadline Period
1 1 3 6
2 2 5 8
```

Scheduling occurs for 24ms

```
0ms: P1
1ms: P1
2ms: idle
3ms: P1
4ms: P1
5ms: P1
6ms: P1
7ms: P1
8ms: P1
9ms: P1
10ms: P1
11ms: P1
12ms: P1
13ms: idle
```

```
14ms: P1
15ms: P1
16ms: idle
```

gantt chart:



LAB PROGRAM 3C:

PROPORTIONAL SCHEDULING:

Code:

```
#include <stdio.h>

#include <stdlib.h>

#include <time.h>

struct Process {

    int pid;

    int tickets;

};

int main() {

    int n, i, totalTickets = 0;

    printf("Enter number of processes: ");

    scanf("%d", &n);

    struct Process p[n];

    for(i = 0; i < n; i++) {

        p[i].pid = i + 1;

        printf("Enter tickets for Process %d: ", p[i].pid);

        scanf("%d", &p[i].tickets);

        totalTickets += p[i].tickets;

    }

    srand(time(NULL));

    printf("\nLottery Scheduling Order:\n")

    for(i = 0; i < 10; i++) { // Simulate 10 CPU allocations

        int winner = rand() % totalTickets + 1;
```

```

int sum = 0, j;

for(j = 0; j < n; j++) {

    sum += p[j].tickets;

    if(winner <= sum) {

        printf("CPU Allocation %d -> Process P%d\n",

            i + 1, p[j].pid);

        break;

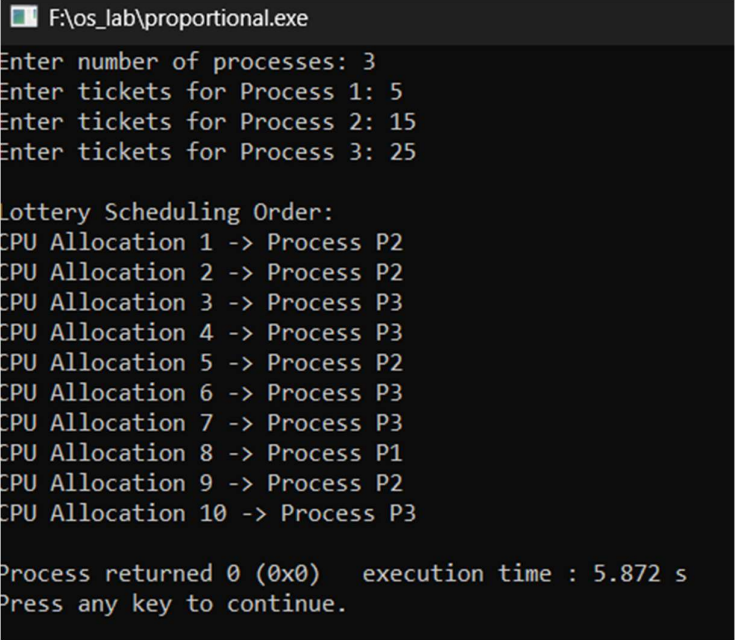
    }

}

return 0;
}

```

Output:



```

F:\os_lab\proportional.exe
Enter number of processes: 3
Enter tickets for Process 1: 5
Enter tickets for Process 2: 15
Enter tickets for Process 3: 25

Lottery Scheduling Order:
CPU Allocation 1 -> Process P2
CPU Allocation 2 -> Process P2
CPU Allocation 3 -> Process P3
CPU Allocation 4 -> Process P3
CPU Allocation 5 -> Process P2
CPU Allocation 6 -> Process P3
CPU Allocation 7 -> Process P3
CPU Allocation 8 -> Process P1
CPU Allocation 9 -> Process P2
CPU Allocation 10 -> Process P3

Process returned 0 (0x0)   execution time : 5.872 s
Press any key to continue.

```

⑫ Proportional scheduling

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

typedef struct {
    int id, share, sum;
} p;

int main() {
    int n, c, total = 0;
    printf("Enter no of processes:");
    scanf("%d", &n);
    p pa[n];
    for (int i = 0; i < n; i++) {
        printf("Share for %d: ", i + 1);
        scanf("%d", &pa[i].share);
        pa[i].id = i + 1;
        pa[i].sum = 0;
        total += pa[i].share;
    }

    printf("Enter no of cycles:");
    scanf("%d", &c);
    srand(time(0));
    while (c--) {
        int win = rand() % total + 1, sum = 0;
        for (int i = 0; i < n; i++) {
            if ((sum + pa[i].share) >= win) {
                printf("P%d selected\n", pa[i].id);
                pa[i].sum++;
                break;
            }
        }
    }
}
```

```
printf("Process schedule:");
for (int i = 0; i < n; i++) {
    printf("%d %d\n", pa[i].id,
           pa[i].sum);
}
```

Output:

Enter no of processes: 3

Share for P1: 5

P2: 15

P3: 25

Enter no of cycles: 12

P2

P3

P3

P1

P2

P3

P2

P3

P3

P1

P2

P3

Execution count:

P1 → 2

P2 → 4

P3 → 6

LAB PROGRAM 4A:

Write a C program to simulate:

PRODUCER CONSUMER:

Code:

```
#include <stdio.h>
#include <stdlib.h>

int mutex = 1;
int full = 0;
int empty = 10;
int x = 0;

int wait(int s)
{
    return (--s);
}

int signal(int s)
{
    return (++s);
}

void producer()
{
    mutex = wait(mutex);
    full = signal(full);
    empty = wait(empty);
    x++;
    printf("Producer produces item %d\n", x);
    mutex = signal(mutex);
}

void consumer()
{
    mutex = wait(mutex);
```

```

    full = wait(full);
    empty = signal(empty);
    printf("Consumer consumes item %d\n", x);
    x--;
    mutex = signal(mutex);
}
int main()
{
    int ch;
    while(1)
    {
        printf("\n1. Producer");
        printf("\n2. Consumer");
        printf("\n3. Exit");
        scanf("%d", &ch);
        switch(ch)
        {
            case 1:
                if((mutex == 1) && (empty != 0))
                    producer();
                else
                    printf("Buffer Full");
                break;
            case 2:
                if((mutex == 1) && (full != 0))
                    consumer();
                else
                    printf("Buffer Empty");
                break;
            case 3:

```

```
        exit(0);  
    }  
}  
return 0;  
}
```

Output:

```
F:\os_lab\producerconsumer.exe  
1. Producer  
2. Consumer  
3. Exit1  
Producer produces item 1  
  
1. Producer  
2. Consumer  
3. Exit2  
Consumer consumes item 1  
  
1. Producer  
2. Consumer  
3. Exit  
2  
Buffer Empty  
1. Producer  
2. Consumer  
3. Exit2  
Buffer Empty  
1. Producer  
2. Consumer  
3. Exit1  
Producer produces item 1  
  
1. Producer  
2. Consumer  
3. Exit1  
Producer produces item 2
```

```

⑧ Producer-consumer
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define n 5
int buffer[n];
int in=0, out=0;
sem_t empty, full;
pthread_mutex_t s;

int produce_item() {
    static int item=1;
    return item++;
}

void consume_item() {
    printf("consumed: %d\n", item);
}

void *producer(void *arg) {
    int item;
    while (1) {
        item = produce_item();
        sem_wait(&empty);
        pthread_mutex_lock(&s);
        buffer[in] = item;
        printf("Produced: %d at position %d\n", item, in);
        in = (in+1)%n;
        pthread_mutex_unlock(&s);
        sem_post(&full);
        sleep(1);
    }
}

void *consumer(void *arg) {
    int c;
    while (1) {
        sem_wait(&full);
        pthread_mutex_lock(&s);
        c = buffer[out];
        printf("consumed: %d from position %d\n", c, out);
        out = (out+1)%n;
        pthread_mutex_unlock(&s);
        sem_post(&empty);
        consume_item(c);
        sleep(2);
    }
}

int main() {
    pthread_t prod, cons;
    sem_init(&empty, 0, n);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&s, NULL);
    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);
    pthread_join(&prod, NULL);
    pthread_join(&cons, NULL);
    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&s);
    return 0;
}

```

output:

```

produced: 1 at pos 0
consumed: 1 from pos 0
consumed: 1

produced: 2 at pos 1
consumed: 1 from pos 3 at pos 2
consumed: 2 from pos 1
consumed: 2

produced: 4 at pos 3
consumed: 1 at pos 5 at pos 4
consumed: 3 from pos 2
consumed: 3

produced: 6 at pos 5
produced: 7 at pos 6
consumed: 4 from pos 3
consumed: 4

```

LAB RPOGRAM 4B:

DINING PHILOSOPHER:

Code:

```
#include <stdio.h>
#include <stdlib.h>
int state[5];
int fork_[5];
#define THINKING 0
#define HUNGRY 1
#define EATING 2
void test(int ph)
{
    if(state[ph] == HUNGRY &&
        state[(ph + 4) % 5] != EATING &&
        state[(ph + 1) % 5] != EATING)
    {
        state[ph] = EATING;
        printf("Philosopher %d takes forks %d and %d\n",
            ph + 1,
            (ph + 4) % 5 + 1,
            ph + 1);
    }
}
void take_fork(int ph)
{
    state[ph] = HUNGRY;
    printf("Philosopher %d is Hungry\n", ph + 1);
    test(ph);
}
void put_fork(int ph)
```



```

{
    state[ph] = THINKING;
    printf("Philosopher %d putting forks %d and %d down\n",
        ph + 1,
        (ph + 4) % 5 + 1,
        ph + 1);
    printf("Philosopher %d is Thinking\n",
        ph + 1);
    test((ph + 4) % 5);
    test((ph + 1) % 5);
}
int main()
{
    int ch, ph;
    for(int i = 0; i < 5; i++)
        state[i] = THINKING;
    while(1)
    {
        printf("\n1.Take Fork");
        printf("\n2.Put Fork");
        printf("\n3.Exit");
        scanf("%d", &ch);
        switch(ch)
        {
            case 1:
                printf("Enter philosopher no:");
                scanf("%d", &ph);
                take_fork(ph - 1);
                break;
            case 2:

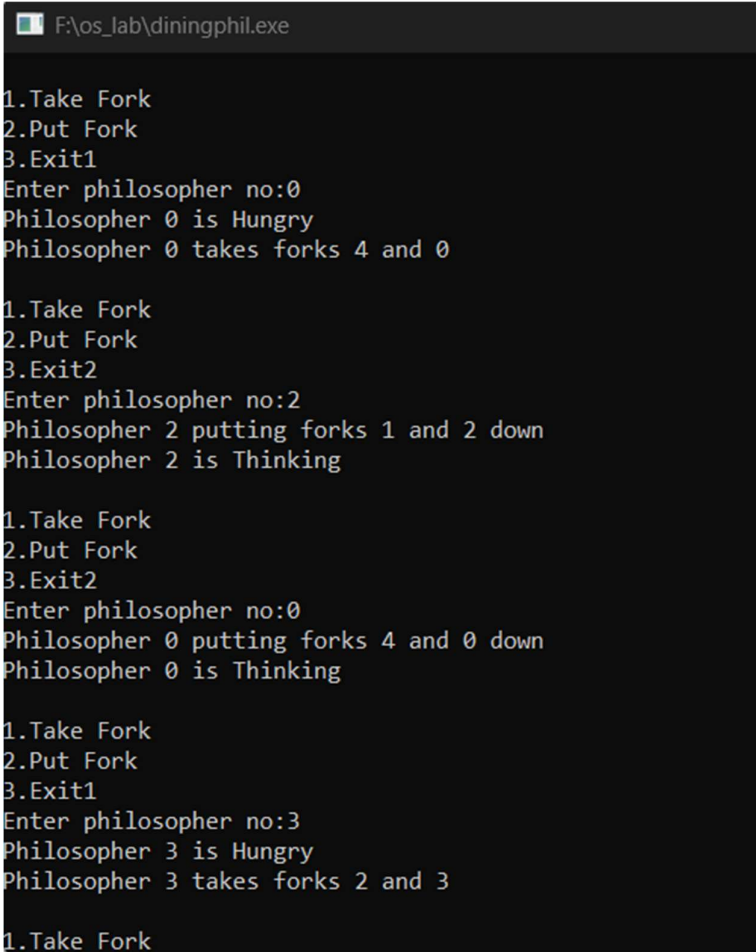
```

```

        printf("Enter philosopher no:");
        scanf("%d", &ph);
        put_fork(ph - 1);
        break;
    case 3:
        exit(0);
    }
}
return 0;
}

```

Output:



```

F:\os_lab\diningphil.exe
1.Take Fork
2.Put Fork
3.Exit1
Enter philosopher no:0
Philosopher 0 is Hungry
Philosopher 0 takes forks 4 and 0

1.Take Fork
2.Put Fork
3.Exit2
Enter philosopher no:2
Philosopher 2 putting forks 1 and 2 down
Philosopher 2 is Thinking

1.Take Fork
2.Put Fork
3.Exit2
Enter philosopher no:0
Philosopher 0 putting forks 4 and 0 down
Philosopher 0 is Thinking

1.Take Fork
2.Put Fork
3.Exit1
Enter philosopher no:3
Philosopher 3 is Hungry
Philosopher 3 takes forks 2 and 3

1.Take Fork

```

Date: / /
Page:

```

⑨ Dining Philosopher
⇒ #include <pthread.h>
#include <semaphore.h>
#include <stdio.h>
#include <unistd.h>
#define n 5

sem_t forks[n];

void *philosopher (void *num) {
    int id = *(int *) num;
    while(1) {
        printf("Philosopher %d is thinking...\n", id);
        sleep(1);
        if(id == n-1) {
            sem_wait(&forks[(id+1)%n]);
            sem_wait(&forks[id]);
        }
        else {
            sem_wait(&forks[id]);
            sem_wait(&forks[(id+1)%n]);
        }
        printf("Phil %d is eating\n", id);
        sleep(2);
        sem_post(&forks[id]);
        sem_post(&forks[(id+1)%n]);
        printf("Phil %d finished eating and put down forks\n", id);
    }
}

```

Date: / /
Page:

```

int main() {
    pthread_t thread_id[n];
    int phil_ids[n];
    for (int i=0; i<n; i++) {
        sem_init(&forks[i], 0, 1);
        phil_ids[i] = i;
    }
    for (int i=0; i<n; i++) {
        pthread_create(&thread_id[i], NULL, phil, &phil_ids[i]);
    }
    for (int i=0; i<n; i++) {
        pthread_join(thread_id[i], NULL);
    }
    for (int i=0; i<n; i++) {
        sem_destroy(&forks[i]);
    }
    return 0;
}

Output:
phil 0 is thinking...
phil 1 is thinking...
phil 2 is thinking...

phil 0 is eating
phil 0 finished eating and put down forks
phil 1 is eating
phil 1 finished eating and put down forks
phil 2 is eating
phil 2 finished eating and put down forks

```

LAB PROGRAM 5A:

Write a C program to simulate:

BANKERS ALGORITHM:

Code:

```
#include <stdio.h>

int main()
{
    int n, m;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[n][m];
    int max[n][m];
    int need[n][m];
    int avail[m];

    printf("Enter Allocation Matrix\n");
    for(int i = 0; i < n; i++)
        for(int j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);

    printf("Enter Max Matrix\n");
    for(int i = 0; i < n; i++)
        for(int j = 0; j < m; j++)
            scanf("%d", &max[i][j]);

    printf("Enter Available Resources\n");
    for(int i = 0; i < m; i++)
        scanf("%d", &avail[i]);

    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < m; j++)
        {
```

```

        need[i][j] =
            max[i][j] - alloc[i][j];
    }
}
int finish[n];
int safe[n];
for(int i = 0; i < n; i++)
    finish[i] = 0;
int count = 0;
while(count < n)
{
    int found = 0;
    for(int i = 0; i < n; i++)
    {
        if(finish[i] == 0)
        {
            int flag = 1;
            for(int j = 0; j < m; j++)
            {
                if(need[i][j] > avail[j])
                {
                    flag = 0;
                    break;
                }
            }
            if(flag)
            {
                safe[count++] = i;
                for(int j = 0; j < m; j++)
                    avail[j] += alloc[i][j];
            }
        }
    }
}

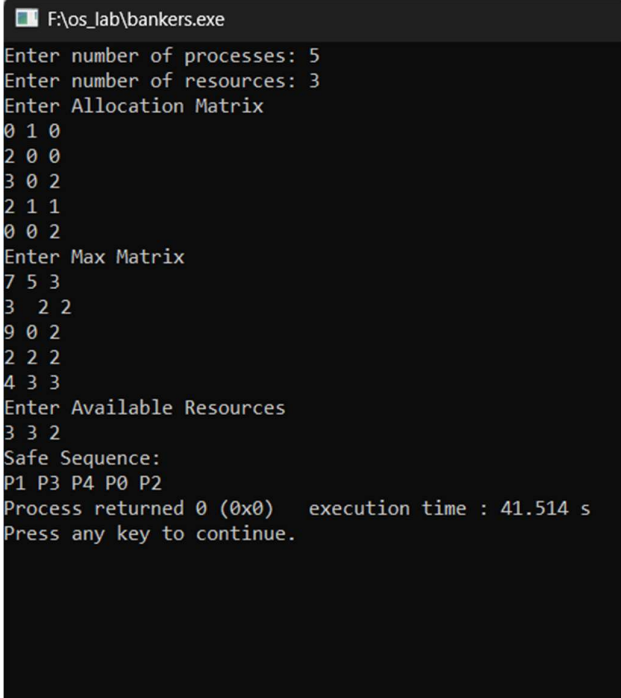
```

```

        finish[i] = 1;
        found = 1;
    }
}
}
if(found == 0)
{
    printf("Unsafe State\n");
    return 0;
}
}
printf("Safe Sequence:\n");
for(int i = 0; i < n; i++)
    printf("P%d ", safe[i]);
return 0;
}

```

Output:



```

F:\os_lab\bankers.exe
Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Max Matrix
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources
3 3 2
Safe Sequence:
P1 P3 P4 P0 P2
Process returned 0 (0x0)   execution time : 41.514 s
Press any key to continue.

```

```

(10) Bankers Algorithm

> #include <stdio.h>
#define max 10
int main() {
    int p, r;
    int alloCmax[max], maxCmax[max],
    need[max][max], availabCmax;
    int finishCmax = {0}, safeSeqCmax;
    printf("Enter number of processes\n");
    scanf("%d", &p);
    printf("Enter no of resources\n");
    scanf("%d", &r);
    printf("Enter alloc matrix\n");
    for(int i=0; i<p; i++) {
        for(int j=0; j<r; j++) {
            scanf("%d", &alloC[i][j]);
        }
    }
    printf("Enter max matrix\n");
    for(int i=0; i<p; i++) {
        for(int j=0; j<r; j++) {
            scanf("%d", &maxC[i][j]);
        }
    }
    printf("Enter avail matrix\n");
    for(int i=0; i<r; i++) {
        scanf("%d", &availC[i]);
    }
    for(int i=0; i<p; i++) {
        for(int j=0; j<r; j++) {
            need[i][j] = maxC[i][j] - alloC[i][j];
        }
    }
    int count=0;

```

```

while (count < p) {
    int found=0;
    for(int i=0; i<p; i++) {
        if(!finishC[i]) {
            for(int j=0; j<r; j++) {
                if(need[i][j] > availC[j])
                    break;
            }
            if(j==r) {
                for(int k=0; k<r; k++)
                    availC[k] =
                    alloC[i][k];
                safeSeqCcount++;
                finishC[i] = 1;
                found = 1;
            }
        }
    }
    if(!found) {
        printf("System is not in safe sequence\n");
        return 0;
    }
    printf("System is in safe sequence\n");
    for(int i=0; i<p; i++) {
        printf("%d ", safeSeqC[i]);
    }
    return 0;
}

```

Date: / /
 Page:

Output:
 Enter no. of process and resource: 5 5
 Enter alloc matrix:

0	1	0
2	0	0
3	0	2
2	1	1
0	0	2

 Enter max matrix:

7	5	3
3	2	2
9	0	2
2	2	2
4	3	3

 Enter avail resource:

3	3	2
---	---	---

 System is in safe sequence
 safe sequence: P₁ P₃ P₄ P₀ P₂

LAB PROGRAM 5B:

DEADLOCK DETECTION:

Code:

```
#include <stdio.h>

int main()
{
    int n, m;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[n][m];
    int max[n][m];
    int need[n][m];
    int avail[m];

    printf("Enter Allocation Matrix\n");
    for(int i = 0; i < n; i++)
        for(int j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);

    printf("Enter Max Matrix\n");
    for(int i = 0; i < n; i++)
        for(int j = 0; j < m; j++)
            scanf("%d", &max[i][j]);

    printf("Enter Available Resources\n");
    for(int i = 0; i < m; i++)
        scanf("%d", &avail[i]);

    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < m; j++)
        {
```



```

        need[i][j] =
            max[i][j] - alloc[i][j];
    }
}
int finish[n];
int safe[n];
for(int i = 0; i < n; i++)
    finish[i] = 0;
int count = 0;
while(count < n)
{
    int found = 0;
    for(int i = 0; i < n; i++)
    {
        if(finish[i] == 0)
        {
            int flag = 1;
            for(int j = 0; j < m; j++)
            {
                if(need[i][j] > avail[j])
                {
                    flag = 0;
                    break;
                }
            }
            if(flag)
            {
                safe[count++] = i;
                for(int j = 0; j < m; j++)
                    avail[j] += alloc[i][j];
            }
        }
    }
}

```

```

        finish[i] = 1;
        found = 1;
    }
}
}
if(found == 0)
{
    printf("Unsafe State\n");
    return 0;
}
}
printf("Safe Sequence:\n");
for(int i = 0; i < n; i++)
    printf("P%d ", safe[i]);
return 0;
}

```

Output:

```

F:\os_lab\deadlockdetection.exe
Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix
0 1 0
2 0 0
3 0 3
2 1 1
0 0 2
Enter Max Matrix
0 0 0
2 0 2
0 0 0
1 0 0
0 0 2
Enter Available Resources
0 0 0
Safe Sequence:
P0 P2 P3 P4 P1
Process returned 0 (0x0) execution time : 36.549 s
Press any key to continue.

```

```

(4) Deadlock detection
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int p, r, i, j;
    printf("Enter no. of process and resources\n");
    scanf("%d %d", &p, &r);
    int alloc[p][r], request[p][r], available[r], work[r];
    bool finish[p];
    printf("Enter alloc matrix\n");
    for(i=0; i<p; i++)
        for(j=0; j<r; j++)
            scanf("%d", &alloc[i][j]);
    printf("Enter request matrix\n");
    for(i=0; i<p; i++)
        for(j=0; j<r; j++)
            scanf("%d", &request[i][j]);
    printf("Enter avail matrix\n");
    for(i=0; i<r; i++)
        scanf("%d", &available[i]);
    for(i=0; i<p; i++)
        work[i] = available[i];
    finish[i] = false;
    for(j=0; j<r; j++)
        if(alloc[i][j] != 0)
            if(finish[i] == false)
                break;
}

bool found = false;
do
{
    found = false;
    for(i=0; i<p; i++)
    {
        if(!finish[i])
        {
            bool canExecute = true;
            for(j=0; j<r; j++)
            {
                if(request[i][j] > work[j])
                {
                    canExecute = false;
                    break;
                }
            }
            if(canExecute)
            {
                for(j=0; j<r; j++)
                    work[j] += alloc[i][j];
                finish[i] = true;
                found = true;
            }
        }
    }
} while(!found);

bool deadlock = false;
for(i=0; i<p; i++)
{
    if(!finish[i])
    {
        deadlock = true;
        printf("process %d is deadlocked\n", i);
    }
}

if(!deadlock)
    printf("no deadlock detected\n");

return 0;
}

```

Output:

Enter number of processes: 5

Enter no of resources: 3

Enter alloc matrix: 0 1 0

2 0 0

3 0 3

2 1 1

0 0 2

Enter request matrix:

0 0 0

2 0 2

0 0 0

1 0 0

0 0 2

Enter available resources: 0 0 0

~~P1 is deadlocked~~

~~P2 is deadlocked~~

~~P3 is deadlocked~~

No dead lock detected

LAB PROGRAM 6:

Write a C program to simulate the following contiguous memory allocation techniques. a) Worst-fit b) Best-fit c) First-fit

Code:

```
#include <stdio.h>

void firstFit(int blockSize[], int m,
              int processSize[], int n)
{
    int allocation[n];
    for(int i = 0; i < n; i++)
        allocation[i] = -1;
    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < m; j++)
        {
            if(blockSize[j] >= processSize[i])
            {
                allocation[i] = j;
                blockSize[j] -= processSize[i];
                break;
            }
        }
    }
    printf("\nFirst Fit\n");
    for(int i = 0; i < n; i++)
    {
        printf("Process %d -> ",
               i + 1);
        if(allocation[i] != -1)
            printf("Block %d\n",
                   allocation[i] + 1);
    }
}
```

```

        else
            printf("Not Allocated\n");
    }
}

void bestFit(int blockSize[], int m,
            int processSize[], int n)
{
    int allocation[n];
    for(int i = 0; i < n; i++)
        allocation[i] = -1;
    for(int i = 0; i < n; i++)
    {
        int bestIdx = -1;
        for(int j = 0; j < m; j++)
        {
            if(blockSize[j] >= processSize[i])
            {
                if(bestIdx == -1 ||
                   blockSize[j] <
                   blockSize[bestIdx])
                {
                    bestIdx = j;
                }
            }
        }
        if(bestIdx != -1)
        {
            allocation[i] = bestIdx;
            blockSize[bestIdx] -= processSize[i];
        }
    }
}

```

```

    }
    printf("\nBest Fit\n");
    for(int i = 0; i < n; i++)
    {
        printf("Process %d -> ",
            i + 1);
        if(allocation[i] != -1)
            printf("Block %d\n",
                allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

void worstFit(int blockSize[], int m,
    int processSize[], int n)
{
    int allocation[n];
    for(int i = 0; i < n; i++)
        allocation[i] = -1;
    for(int i = 0; i < n; i++)
    {
        int worstIdx = -1;
        for(int j = 0; j < m; j++)
        {
            if(blockSize[j] >= processSize[i])
            {
                if(worstIdx == -1 ||
                    blockSize[j] >
                    blockSize[worstIdx])
                {

```

```

        worstIdx = j;
    }
}
}
if(worstIdx != -1)
{
    allocation[i] = worstIdx;
    blockSize[worstIdx] -= processSize[i];
}
}

printf("\nWorst Fit\n");
for(int i = 0; i < n; i++)
{
    printf("Process %d -> ",
        i + 1);
    if(allocation[i] != -1)
        printf("Block %d\n",
            allocation[i] + 1);
    else
        printf("Not Allocated\n");
}
}

int main()
{
    int m, n;
    printf("Enter number of blocks: ");
    scanf("%d", &m);
    int blockSize[m];
    printf("Enter block sizes:\n");

```

```

for(int i = 0; i < m; i++)
    scanf("%d", &blockSize[i]);
printf("Enter number of processes: ");
scanf("%d", &n);
int processSize[n];
printf("Enter process sizes:\n");
for(int i = 0; i < n; i++)
    scanf("%d", &processSize[i]);
int block1[m], block2[m], block3[m];
for(int i = 0; i < m; i++)
{
    block1[i] = blockSize[i];
    block2[i] = blockSize[i];
    block3[i] = blockSize[i];
}
firstFit(block1, m, processSize, n);
bestFit(block2, m, processSize, n);
worstFit(block3, m, processSize, n);
return 0;
}
Output:

```

```

Select F:\os_lab\memoryalloc.exe
Enter number of blocks: 5
Enter block sizes:
100 500 200 300 600
Enter number of processes: 4
Enter process sizes:
212 417 112 426

First Fit
Process 1 -> Block 2
Process 2 -> Block 5
Process 3 -> Block 2
Process 4 -> Not Allocated

Best Fit
Process 1 -> Block 4
Process 2 -> Block 2
Process 3 -> Block 3
Process 4 -> Block 5

Worst Fit
Process 1 -> Block 5
Process 2 -> Block 2
Process 3 -> Block 5
Process 4 -> Not Allocated

Process returned 0 (0x0)   execution time : 33.814 s
Press any key to continue.

```



```

13) Contiguous memory allocation technique

=> #include <stdio.h>

void firstFit(int *blocks, int m, int *processes,
             int n) {
    int alloc[50];
    int temp[50];
    for (int i=0; i<m; i++) temp[i] = blocks[i];
    for (int i=0; i<n; i++) alloc[i] = -1;
    for (int i=0; i<n; i++) {
        for (int j=0; j<m; j++) {
            if (temp[j] >= processes[i]) {
                alloc[j] = i;
                temp[j] -= processes[i];
                break;
            }
        }
        printf("In First Fit allocation: (%i)\n", i);
        display(alloc, processes, n);
    }
}

void bestFit(int *blocks, int m, int *processes,
            int n) {
    int alloc[50], temp[50];
    for (int i=0; i<m; i++) temp[i] = blocks[i];
    for (int i=0; i<n; i++) alloc[i] = -1;
    for (int i=0; i<n; i++) {
        for (int j=0; j<m; j++) {
            if (temp[j] >= processes[i]) {
                if (bestIndex == -1 || temp[j] < temp[bestIndex]) {
                    bestIndex = j;
                }
            }
        }
        if (bestIndex != -1) {
            alloc[bestIndex] = i;
            temp[bestIndex] -= processes[i];
        }
        printf("In Best Fit allocation: (%i)\n", i);
        display(alloc, processes, n);
    }
}

void worstFit(int *blocks, int m, int *processes,
             int n) {
    int alloc[50], temp[50];
    for (int i=0; i<m; i++) temp[i] = blocks[i];
    for (int i=0; i<n; i++) alloc[i] = -1;
    for (int i=0; i<n; i++) {
        for (int j=0; j<m; j++) {
            if (temp[j] >= processes[i]) {
                if (worstIndex == -1 || temp[j] > temp[worstIndex]) {
                    worstIndex = j;
                }
            }
        }
        if (worstIndex != -1) {
            alloc[worstIndex] = i;
            temp[worstIndex] -= processes[i];
        }
        printf("In worst fit allocation: (%i)\n", i);
        display(alloc, processes, n);
    }
}

```

```

void display(int *alloc, int *processes, int n) {
    printf("In processes No. < Process size < Block\n\n");
    for (int i=0; i<n; i++) {
        printf("%d < (%i < (%i)\n", i, processes[i], alloc[i]);
        if (alloc[i] == -1) {
            printf("Not allocated\n");
        }
    }
}

int main() {
    int m, n, blocks[50], processes[50];
    printf("Enter no. of blocks, processes and\n\n");
    printf("size of processes:\n");
    scanf("%d", &m);
    for (int i=0; i<m; i++) {
        scanf("%d", &blocks[i]);
        printf("Enter no. of processes and\n\n");
        printf("size of processes:\n");
        scanf("%d", &n);
        for (int i=0; i<n; i++) {
            scanf("%d", &processes[i]);
            printf("In Memory allocation techniques\n\n");
            1. First Fit 2. Best Fit 3. Worst Fit
            scanf("%d", &ch);
            switch (ch) {
                case 1: firstFit(blocks, m, processes, n); break;
                case 2: bestFit(blocks, m, processes, n); break;
                case 3: worstFit(blocks, m, processes, n); break;
                default: printf("Invalid choice!!\n");
            }
        }
    }
}

```

Output:

Enter no. of memory blocks: 5
Enter sizes of memory blocks:
100 500 200 300 600

Enter no. of processes: 4
Enter sizes of processes:
212 419 112 426

Memory allocation techniques

1. First Fit
2. Best Fit
3. Worst Fit

1. First Fit Allocation -

Process No.	Process size	Block No.
1	212	2
2	419	5
3	112	2
4	426	Not allocated

2. Best Fit allocation:

Process No.	Process size	Block No.
1	212	4
2	419	2
3	112	3
4	426	5

3. Worst Fit allocation:

Process No.	Process size	Block No.
1	212	5
2	419	2
3	112	5
4	426	Not allocated

LAB PROGRAM 7:

Write a C program to simulate page replacement algorithms. a) FIFO b) LRU c) Optimal

Code:

```
#include <stdio.h>

void fifo(int pages[], int n, int f)
{
    int frames[20];
    int index = 0, faults = 0;
    for(int i = 0; i < f; i++)
        frames[i] = -1;
    for(int i = 0; i < n; i++)
    {
        int found = 0;
        for(int j = 0; j < f; j++)
        {
            if(frames[j] == pages[i])
            {
                found = 1;
                break;
            }
        }
        if(!found)
        {
            frames[index] = pages[i];
            index = (index + 1) % f;
            faults++;
        }
    }
    printf("FIFO Page Faults = %d\n", faults);
}
```

```

void lru(int pages[], int n, int f)
{
    int frames[20], time[20];
    int faults = 0;
    for(int i = 0; i < f; i++)
    {
        frames[i] = -1;
        time[i] = -1;
    }
    for(int i = 0; i < n; i++)
    {
        int found = 0;
        for(int j = 0; j < f; j++)
        {
            if(frames[j] == pages[i])
            {
                found = 1;
                time[j] = i;
                break;
            }
        }
        if(!found)
        {
            int pos = -1;
            for(int j = 0; j < f; j++)
            {
                if(frames[j] == -1)
                {
                    pos = j;
                    break;
                }
            }
        }
    }
}

```

```

        }
    }
    if(pos == -1)
    {
        pos = 0;
        for(int j = 1; j < f; j++)
        {
            if(time[j] < time[pos])
                pos = j;
        }
    }
    frames[pos] = pages[i];
    time[pos] = i;
    faults++;
}
}
printf("LRU Page Faults = %d\n", faults);
}

void optimal(int pages[], int n, int f)
{
    int frames[20];
    int faults = 0;
    for(int i = 0; i < f; i++)
        frames[i] = -1;
    for(int i = 0; i < n; i++)
    {
        int found = 0;
        for(int j = 0; j < f; j++)
        {
            if(frames[j] == pages[i])

```

```

        {
            found = 1;
            break;
        }
    }
    if(!found)
    {
        int pos = -1;

        for(int j = 0; j < f; j++)
        {
            if(frames[j] == -1)
            {
                pos = j;
                break;
            }
        }
        if(pos == -1)
        {
            int farthest = -1;
            for(int j = 0; j < f; j++)
            {
                int k;
                for(k = i + 1; k < n; k++)
                {
                    if(frames[j] == pages[k])
                        break;
                }
                if(k == n)
                {

```

```

        pos = j;
        break;
    }
    if(k > farthest)
    {
        farthest = k;
        pos = j;
    }
}

frames[pos] = pages[i];
faults++;
}
}

printf("Optimal Page Faults = %d\n", faults);
}

int main()
{
    int n, f, ch;
    int pages[50];
    printf("Enter number of pages: ");
    scanf("%d", &n);
    printf("Enter reference string:\n");
    for(int i = 0; i < n; i++)
        scanf("%d", &pages[i]);
    printf("Enter number of frames: ");
    scanf("%d", &f);
    printf("\n1. FIFO");
    printf("\n2. LRU");
    printf("\n3. Optimal");

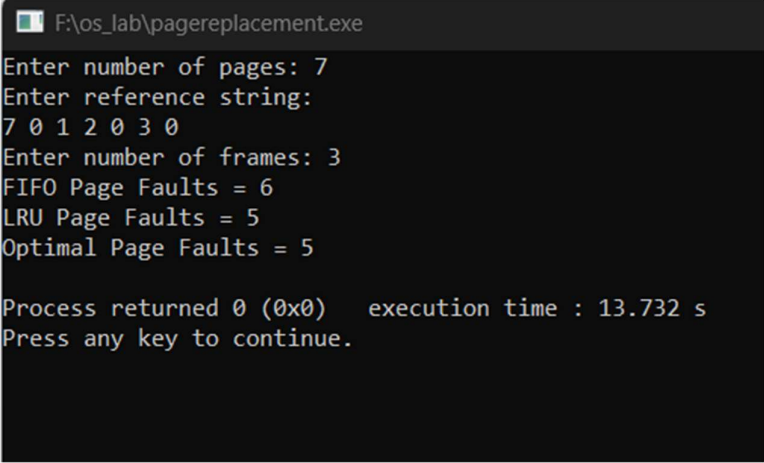
```

```

printf("\nEnter choice: ");
scanf("%d", &ch);
switch(ch)
{
    case 1:
        fifo(pages, n, f);
        break;
    case 2:
        lru(pages, n, f);
        break;
    case 3:
        optimal(pages, n, f);
        break;
    default:
        printf("Invalid Choice");
}
return 0;
}

```

Output:



```

F:\os_lab\pagereplacement.exe
Enter number of pages: 7
Enter reference string:
7 0 1 2 0 3 0
Enter number of frames: 3
FIFO Page Faults = 6
LRU Page Faults = 5
Optimal Page Faults = 5

Process returned 0 (0x0)   execution time : 13.732 s
Press any key to continue.

```

```

1) Page replacement algorithm
#include <iostream>

void FIFO (int *pages, int n, int frames) {
    int frame[frames], fault=0, index=0, found;
    for (int i=0; i<frames; i++) frame[i]=-1;
    for (int i=0; i<n; i++) {
        found=0;
        for (int j=0; j<frames; j++)
            if (frame[j]==pages[i]) {
                found=1; break;
            }
        if (!found) {
            frame[index]=pages[i];
            index=(index+1)%frames;
            fault++;
        }
    }
}

// print LRU page faults = %d\n, fault;
// print LRU page faults = %d\n, fault;

void LRU (int *pages, int n, int frames) {
    int frame[frames], time[frames], fault=0, count=1;
    for (int i=0; i<frames; i++) {
        frame[i]=-1;
        time[i]=0;
    }
    for (int i=0; i<n; i++) {
        int found=0;
        for (int j=0; j<frames; j++) {
            if (frame[j]==pages[i]) {
                found=1;
                time[j]=count;
            }
        }
        if (!found) {
            // find the index of the least recently used page
            int minIndex=-1;
            for (int k=0; k<frames; k++) {
                if (minIndex==-1 || time[k]<time[minIndex])
                    minIndex=k;
            }
            frame[minIndex]=pages[i];
            time[minIndex]=count;
            fault++;
        }
        count++;
    }
}

// print LRU page faults = %d\n, fault;
// print LRU page faults = %d\n, fault;

void optimal (int *pages, int n, int frames) {
    int frame[frames], fault=0, count=1;
    for (int i=0; i<frames; i++) {
        frame[i]=-1;
        time[i]=0;
    }
    for (int i=0; i<n; i++) {
        int found=0;
        for (int j=0; j<frames; j++) {
            if (frame[j]==pages[i]) {
                found=1;
                time[j]=count;
            }
        }
        if (!found) {
            // find the index of the least recently used page
            int minIndex=-1;
            for (int k=0; k<frames; k++) {
                if (minIndex==-1 || time[k]<time[minIndex])
                    minIndex=k;
            }
            frame[minIndex]=pages[i];
            time[minIndex]=count;
            fault++;
        }
        count++;
    }
}

// print LRU page faults = %d\n, fault;
// print LRU page faults = %d\n, fault;

int main() {
    int n, frames;
    cout<<"Enter no. of pages:"<<endl;
    cin>>n;
    cout<<"Enter page sequence string:"<<endl;
    string s;
    cin>>s;
    int pages[n];
    for (int i=0; i<n; i++)
        pages[i]=s[i]-'0';
    cout<<"Enter no. of frames:"<<endl;
    cin>>frames;
    FIFO(pages, n, frames);
    LRU(pages, n, frames);
    optimal(pages, n, frames);
    return 0;
}

```

```

// print LRU page faults = %d\n, fault;
// print LRU page faults = %d\n, fault;

void optimal (int *pages, int n, int frames) {
    int frame[frames], fault=0, count=1;
    for (int i=0; i<frames; i++) {
        frame[i]=-1;
        time[i]=0;
    }
    for (int i=0; i<n; i++) {
        int found=0;
        for (int j=0; j<frames; j++) {
            if (frame[j]==pages[i]) {
                found=1;
                time[j]=count;
            }
        }
        if (!found) {
            // find the index of the least recently used page
            int minIndex=-1;
            for (int k=0; k<frames; k++) {
                if (minIndex==-1 || time[k]<time[minIndex])
                    minIndex=k;
            }
            frame[minIndex]=pages[i];
            time[minIndex]=count;
            fault++;
        }
        count++;
    }
}

// print LRU page faults = %d\n, fault;
// print LRU page faults = %d\n, fault;

int main() {
    int n, frames;
    cout<<"Enter no. of pages:"<<endl;
    cin>>n;
    cout<<"Enter page sequence string:"<<endl;
    string s;
    cin>>s;
    int pages[n];
    for (int i=0; i<n; i++)
        pages[i]=s[i]-'0';
    cout<<"Enter no. of frames:"<<endl;
    cin>>frames;
    FIFO(pages, n, frames);
    LRU(pages, n, frames);
    optimal(pages, n, frames);
    return 0;
}

```

FIFO (pages, n, frames);
 LRU (pages, n, frames);
 OPTIMAL (pages, n, frames);
 return 0;

output:
 Enter no of pages : 13
 Enter page sequence string:
 7 0 1 2 0 3 0 4 2 3 0 3 2
 Enter no of frames

FIFO page faults = 7
 LRU page faults = 6
 optimal page faults = 6

FIFO:

0	3		
2	2	2	
1	1	1	
0	0	0	
7	3	3	

 H F - -

optimal:

3			
2	2	2	
1	1	1	
0	0	0	
7	3	3	

 H F - -

LRU:

0	3		
2	2		
1	1		
0	0		
7	3		

 H F - -

as 1 is least recently used.